

- constraint
 - difference, 627
 - equality, 632 *ex.*
 - linear, 851, 853–854
 - nonnegativity, 854
- constraint graph, 628–630
- contain, in a path, 1165
- continuous master theorem, 112
 - proof of, 107–115
- continuous uniform probability distribution, 1187
- contraction
 - of a dynamic table, 465–470
 - of an undirected graph by an edge, 1168
- contraction algorithm, 701 *pr.*
- control instructions, 26
- convergence property, 611, 634–635
- convergent series, 1140
- converting binary to decimal, 910 *ex.*
- convex body, 1032
- convex function, 1025–1027, 1194
 - α -strongly convex, 1041
- convex set, 675 *ex.*
- convolution ($\otimes$), 880
- convolution theorem, 892
- copy instruction, 26
- correctness of an algorithm, 6
- corresponding flow network for bipartite matching, 694
- countably infinite set, 1156
- counter, *see* binary counter
- counting, 1178–1184
 - probabilistic, 153 *pr.*
- counting sort, 208–211
 - in computing suffix arrays, 992
 - in radix sort, 213
- COUNTING-SORT, 209
- coupon collector's problem, 144
- cover
 - path, 698 *pr.*
 - by a subfamily, 1116
 - vertex, 1084, 1106
- credit, 453
- critical edge, 690
- critical path
 - in a dag, 619
 - of a PERT chart, 617
 - of a task-parallel trace, 757
- cross a cut, 587, 701 *pr.*
- cross edge, 569
- cryptographic hash function, 291
- cryptosystem, 936–942
- cubic spline, 847 *pr.*
- currency exchange, 641 *pr.*
- curve fitting, 841–845
- cut
 - capacity of, 682
 - of a flow network, 682–685
 - global, 701 *pr.*
 - minimum, 682
 - net flow across, 682
 - of an undirected graph, 587
 - weight of, 1124 *ex.*
- CUT-ROD, 366
- cycle of a graph, 1165–1166
 - hamiltonian, 1043, 1056, 1085–1090
 - minimum mean-weight, 642 *pr.*
 - negative-weight, *see* negative-weight cycle
 - and shortest paths, 607–608
- cycle cover, 741 *pr.*
- cyclic group, 932
- dag, *see* directed acyclic graph
- DAG-SHORTEST-PATHS, 617
- d -ary heap, 179 *pr.*
 - in shortest-paths algorithms, 668 *pr.*
- data-movement instructions, 26
- data-parallel model, 789
- data science, 14–15
- data structure, 9, 249–359, 477–545
 - AA-trees, 358
 - augmentation of, 480–496
 - AVL trees, 357 *pr.*, 358
 - binary search trees, 312–330
 - bit vectors, 274 *ex.*
 - B-trees, 497–519
 - deques, 258 *ex.*
 - dictionaries, 249
 - direct-address tables, 273–275
 - for disjoint sets, 520–545
 - for dynamic graphs, 479
 - dynamic sets, 249–251
 - dynamic trees, 478
 - exponential search trees, 226, 478
 - Fibonacci heaps, 478
 - fusion trees, 226, 478

- data structure, *continued*
 - hash tables, 275–282
 - heaps, 161–181
 - interval trees, 489–495
 - k -neighbor trees, 358
 - left-leaning red-black binary search trees, 358
 - linked lists, 258–264
 - order-statistic trees, 480–486
 - persistent, 355 pr., 478
 - potential of, 456
 - priority queues, 172–178
 - queues, 254, 256–257
 - radix trees, 327 pr.
 - red-black trees, 331–359
 - rooted trees, 265–268
 - scapegoat trees, 358
 - on secondary storage, 498–501
 - skip lists, 359
 - splay trees, 359, 478
 - stacks, 254–255
 - treaps, 358
 - 2-3-4 trees, 502, 518 pr.
 - 2-3 trees, 358, 519
 - van Emde Boas trees, 478
 - weight-balanced trees, 358
- data type, 26
- decision by an algorithm, 1053
- decision problem, 1045, 1049
 - and optimization problems, 1045
- decision tree, 206–207, 219 pr.
- decision variable, 851
- DECREASE-KEY, 173
- decrementing, 22
- decryption, 936
- default vertex labeling, 724
- degree
 - minimum, of a B-tree, 502
 - of a node, 1173
 - of a polynomial, 65, 877
 - of a vertex, 1165
- degree-bound, 877
- DELETE, 250
- DELETE-LARGER-HALF, 460 ex.
- deletion
 - from binary search trees, 322–325, 326 ex.
 - from B-trees, 513–516
 - from chained hash tables, 278
 - from direct-address tables, 274
 - from dynamic tables, 465–470
 - from hash tables with linear probing, 302–303
 - from heaps, 178 ex.
 - from interval trees, 491
 - from linked lists, 261
 - from open-address hash tables, 294–295
 - from order-statistic trees, 484–485
 - from queues, 256
 - from red-black trees, 346–355
 - from stacks, 254
- DeMorgan's laws
 - for propositional logic, 1078
 - for sets, 1155, 1158 ex.
- dense graph, 549
 - ϵ -dense, 668 pr.
- dense matrix, 81
- density
 - of prime numbers, 943
 - of a rod, 372 ex.
- dependence
 - and indicator random variables, 131
 - linear, 1220
 - see also* independence
- depth
 - average, of a node in a randomly built binary search tree, 328 pr.
 - of a circuit, 894
 - of a node in a rooted tree, 1173
 - of quicksort recursion tree, 191 ex.
 - of a stack, 202 pr.
- depth-determination problem, 542 pr.
- depth-first forest, 564
- depth-first search, 563–572
 - in finding articulation points, bridges, and biconnected components, 582 pr.
 - in finding strongly connected components, 576–581
 - in the Hopcroft-Karp algorithm, 711
 - in topological sorting, 573–576
- depth-first tree, 564
- deque, 258 ex.
- DEQUEUE, 257
- derivative of a series, 1142
- descendant, 1172
- destination vertex, 605
- det, 1221

- determinacy race, 765–768
- determinant, 1221
- deterministic algorithm, 135
 - parallel, 765
- DETERMINISTIC-SEARCH, 154 pr.
- DFS, 565
 - see also* depth-first search
- DFS-VISIT, 565
- DFT, 888
- diagonal matrix, 1215
- diameter
 - of a network, 646
 - of a tree, 563 ex.
- dictionary, 249
- difference
 - of sets ($-$), 1154
 - symmetric, 706
- difference constraints, 626–632
- differentiation of a series, 1142
- digital signature, 938
- digraph, *see* directed graph
- DIJKSTRA, 620
- Dijkstra’s algorithm, 620–626
 - for all-pairs shortest paths, 646, 666
 - with edge weights in a range, 626 ex.
 - implemented with a Fibonacci heap, 623–624
 - implemented with a min-heap, 623
 - with integer edge weights, 625–626 ex.
 - in Johnson’s algorithm, 664
 - similarity to breadth-first search, 624, 625 ex.
 - similarity to Prim’s algorithm, 624
- d -independent family of hash functions, 288
- DIRECT-ADDRESS-DELETE, 274
- direct addressing, 273–275
- DIRECT-ADDRESS-INSERT, 274
- DIRECT-ADDRESS-SEARCH, 274
- direct-address table, 273–275
- directed acyclic graph (dag), 1167
 - and back edges, 573
 - and component graphs, 578
 - and hamiltonian paths, 1060 ex.
 - longest simple path in, 407 pr.
 - for representing a parallel computation, 754
 - single-source shortest-paths algorithm for, 616–619
 - topological sort of, 573–576
- directed equality subgraph, 727
- directed graph, 1164
 - all-pairs shortest paths in, 646–669
 - constraint graph, 628
 - Euler tour of, 583 pr., 1043
 - hamiltonian cycle of, 1043
 - incidence matrix of, 553 ex.
 - and longest paths, 1042
 - path cover of, 698 pr.
 - PERT chart, 617, 619 ex.
 - semiconnected, 581 ex.
 - shortest path in, 604
 - single-source shortest paths in, 604–645
 - singly connected, 572 ex.
 - square of, 553 ex.
 - transitive closure of, 659
 - transpose of, 553 ex.
 - universal sink in, 553 ex.
 - see also* directed acyclic graph, graph, network
- directed version of an undirected graph, 1166
- dirty area, 222 pr.
- discovered vertex, 554, 564
- discovery time, 565
- discrete Fourier transform, 888
- discrete logarithm, 933
- discrete logarithm theorem, 933
- discrete probability distribution, 1186
- discrete random variable, 1191–1196
- disjoint-set data structure, 520–545
 - analysis of, 534–540
 - in connected components, 521–523
 - in depth determination, 542 pr.
 - disjoint-set-forest implementation of, 527–531
 - in Kruskal’s algorithm, 593
 - linear-time special case of, 545
 - linked-list implementation of, 523–527
 - lower bound for, 545
 - in offline lowest common ancestors, 543 pr.
 - in offline minimum, 541 pr.
- disjoint-set forest, 527–531
 - analysis of, 534–540
 - rank properties of, 533–534, 540 ex.
 - see also* disjoint-set data structure
- disjoint sets, 1156
- disjunctive normal form, 1078

- disk drive, 498–500
 - see also* secondary storage
- DISK-READ, 500
- DISK-WRITE, 500
- dissimilarity, 1006
- distance
 - edit, 409 pr.
 - Manhattan, 244 pr.
 - of a shortest path (δ), 558
- distributed memory, 748
- distribution
 - binomial, *see* binomial distribution
 - continuous uniform, 1187
 - discrete, 1186
 - geometric, *see* geometric distribution
 - of inputs, 128, 134
 - of prime numbers, 943
 - probability, 218 ex., 1185
 - uniform, 1186
- distributive laws for sets, 1155
- divergent series, 1140
- divide-and-conquer method, 34, 76
 - analysis of, 39–41, 90–119
 - for binary search, 44 ex.
 - for conversion of binary to decimal, 910 ex.
 - for fast Fourier transform, 888–891, 895
 - for matrix inversion, 834–837
 - for matrix multiplication, 81–90, 770–775, 783 pr.
 - for merge sort, 34–44, 775–782
 - for multiplication, 899 pr.
 - for quicksort, 182–204
 - relation to dynamic programming, 362
 - for selection, 230–243
 - solving recurrences for, 90–119
 - for Strassen’s algorithm, 85–90, 773–774
- divide instruction, 26
- divides relation ($\mid$), 904
- divide step, in divide-and-conquer, 34, 76
- division method, 284, 292 ex.
- division theorem, 905
- divisor, 904
 - common, 906
 - see also* greatest common divisor
- DNA, 6, 393–394, 409 pr.
- DNF (disjunctive normal form), 1078
- does-not-divide relation ($\nmid$), 904
- Dog River, 717
- dolphins, allowing to vote, 850
- domain, 1161
- double hashing, 295–297, 301 ex.
- doubly linked list, 258–259, 264 ex.
 - circular, with a sentinel, 262
- downto**, in pseudocode, 22
- d -regular graph, 716 ex., 740 pr.
- driving function, 101
- duality, 734, 866–873, 874 pr.
 - weak, 868–869, 874 pr.
- dual linear program, 866
- dummy key, 400
- dynamic graph, 523
 - all-pairs shortest paths algorithms for, 669
 - data structures for, 479
 - minimum-spanning-tree algorithm for, 599 ex.
 - transitive closure of, 667 pr., 669
- dynamic graph algorithm, 817
- dynamic multiset, 460 ex.
- dynamic order statistics, 480–486
- dynamic-programming method, 362–416
 - for activity selection, 424 ex.
 - for all-pairs shortest paths, 648–659
 - for bitonic euclidean traveling-salesperson problem, 407 pr.
 - bottom-up, 368
 - for breaking a string, 412 pr.
 - compared with greedy method, 384–385, 393 ex., 421, 426–430
 - for edit distance, 409 pr.
 - elements of, 382–393
 - for Floyd-Warshall algorithm, 655–659
 - for inventory planning, 414 pr.
 - for longest common subsequence, 393–399
 - for longest palindrome subsequence, 407 pr.
 - for longest simple path in a weighted directed acyclic graph, 407 pr.
 - for matrix-chain multiplication, 373–382 and memoization, 390–392
 - for optimal binary search trees, 400–407
 - optimal substructure in, 382–387
 - overlapping subproblems in, 387–390
 - for printing neatly, 408 pr.
 - reconstructing an optimal solution in, 390
 - relation to divide-and-conquer, 362
 - for rod cutting, 363–373
 - for seam carving, 412 pr.

- dynamic-programming method, *continued*
 - for signing free agents, 414 pr.
 - top-down with memoization, 368
 - for transitive closure, 659–661
 - for Viterbi algorithm, 411 pr.
 - for the 0-1 knapsack problem, 430 ex.
- dynamic set, 249–251
 - see also* data structure
- dynamic table, 460–471
 - analyzed by accounting method, 463
 - analyzed by aggregate analysis, 462–463
 - analyzed by potential method, 463–470
 - load factor of, 461
- dynamic tree, 478
- $E[\]$, *see* expected value
- e (base of the natural logarithm), 65
- edge, 1164
 - antiparallel, 673–674
 - attributes of, 552
 - back, 569
 - bridge, 582 pr.
 - capacity of, 671
 - classification in breadth-first search, 581 pr.
 - classification in depth-first search, 569–570, 571 ex.
 - critical, 690
 - cross, 569
 - forward, 569
 - light, 587
 - negative-weight, 606–607
 - residual, 678
 - safe, 587
 - tree, 561, 564, 569
 - weight of, 551
- edge connectivity, 692 ex.
- edge set, 1164
- edit distance, 409 pr.
- Edmonds-Karp algorithm, 689–691
- elementary event, 1185
- elementary insertion, 461
- element of a set ($\in$), 1153
- ellipsoid algorithm, 857
- elliptic-curve factorization method, 956
- elseif**, in pseudocode, 22 n.
- else**, in pseudocode, 22
- empty language ($\emptyset$), 1052
- empty set ($\emptyset$), 1153
- empty set laws, 1154
- empty stack, 255
- empty string (ε), 959, 1052
- empty tree, 1173
- encoding of problem instances, 1049–1052
- encryption, 936
- endpoint of an interval, 489
- ENQUEUE, 257
- entering a vertex, 1164
- entropy function, 1182
- epoch, 805
- ϵ -dense graph, 668 pr.
- ϵ -universal family of hash functions, 287, 292 ex.
- equality
 - of functions, 1162
 - linear, 853
 - of sets, 1153
- equality constraint, 632 ex.
- equality subgraph, 724
 - directed, 727
- equations and asymptotic notation, 58–59
- equivalence class, 1159
 - modulo n ($[a]_n$), 905
- equivalence, modular ($= (\bmod n)$), 64
- equivalence relation, 1159
- error bound, 1027
- error**, in pseudocode, 24
- escape problem, 697 pr.
- EUCLID, 912
- Euclid's algorithm, 911–916, 954 pr.
- euclidean norm ($\| \ \|$), 1219
- Euler's constant, 921
- Euler's phi function, 920
- Euler's theorem, 932, 953 ex.
- Euler tour, 583 pr., 740 pr.
 - and hamiltonian cycles, 1043
- evaluation of a polynomial, 46 pr., 879, 884 ex.
 - derivatives of, 900 pr.
 - at multiple points, 900 pr.
- event, 1185
- event-driven simulation, 173, 181
- EXACT-SUBSET-SUM, 1125
- example, in clustering, 1006
- exclusion and inclusion, 1158 ex.
- execute a subroutine, 29 n.
- expansion of a dynamic table, 461–465
- expectation, *see* expected value

- expected competitive ratio, 808
- expected running time, 32, 129
- expected value, 1192–1194
 - of a binomial distribution, 1198
 - of a geometric distribution, 1197
 - of an indicator random variable, 130
- explored edge, 565
- exponential function, 65–66
- exponential search tree, 226, 478
- exponentiation
 - of logarithms, 66
 - modular, 934–935
- exponentiation instruction, 27
- EXTENDED-BOTTOM-UP-CUT-ROD, 372
- EXTENDED-EUCLID, 914
- EXTEND-SHORTEST-PATHS, 650
- external node, 1172
- external path length, 1175 ex.
- extracting the maximum key
 - from d -ary heaps, 179 pr.
 - from max-heaps, 174
- extracting the minimum key
 - from Young tableaux, 179 pr.
- EXTRACT-MAX, 173–174
- EXTRACT-MIN, 173
- factor, 904
 - twiddle, 891
- factorial function (!), 67–68
- factorization, 956
 - unique, 909
- failure, in a Bernoulli trial, 1196
- fair coin, 1186
- family of hash functions, 286–288, 292 ex.
- fan-out, 1066
- Farkas’s lemma, 869
- FASTER-APSP, 653, 655 ex.
- fast Fourier transform (FFT), 877–902
 - circuit for, 894–897
 - multidimensional, 899 pr.
 - recursive implementation of, 888–891
 - using modular arithmetic, 901 pr.
- feasibility problem, 627, 873 pr.
- feasible linear program, 854
- feasible region, 854
- feasible solution, 627, 854
- feasible vertex labeling, 724, 742 pr.
- feature vector, 1006
- Fermat’s theorem, 932
- FFT, 890
 - see also* fast Fourier transform
- FFTW, 902
- FIB, 751
- Fibonacci heap, 478
 - in Dijkstra’s algorithm, 623–624
 - in Johnson’s algorithm, 666
 - in Prim’s algorithm, 597
- Fibonacci numbers, 69, 70 ex., 121 pr.
 - computation of, 750–753, 954 pr.
- FIFO, *see* first-in, first-out; queue
- final-state function, 968
- FIND-AUGMENTING-PATH, 738
- FIND-DEPTH, 542 pr.
- find path, 528
- FIND-POM, 496 pr.
- FIND-SET, 521
 - disjoint-set-forest implementation of, 530, 544
 - linked-list implementation of, 523
- FIND-SPLIT-POINT, 778
- finished vertex, 564
- finish time
 - in activity selection, 418
 - in depth-first search, 565
 - and strongly connected components, 578
- finite automaton, 967–975
- FINITE-AUTOMATON-MATCHER, 971
- finite group, 917
- finite sequence, 1162
- finite set, 1156
- finite sum, 1140
- first-fit heuristic, 1131 pr.
- first-in, first-out (FIFO), 254, 803–804, 814 ex.
 - implemented with a priority queue, 178 ex.
 - see also* queue
- fixed-length code, 432
- floating-point data type, 26
- floor function ($\lfloor \rfloor$), 63
 - in recurrences, 116–117
- floor instruction, 26
- flow, 671–676
 - aggregate, 864
 - augmentation of, 678
 - blocking, 702
 - cancellation of, 679
 - integer-valued, 695

- flow, *continued*
 - net, across a cut, 682
 - value of, 672
- flow conservation, 672
- flow network, 671–676
 - corresponding to a bipartite graph, 694
 - cut of, 682–685
 - with multiple sources and sinks, 674
- FLOYD-WARSHALL, 657
- FLOYD-WARSHALL', 661 *ex.*
- Floyd-Warshall algorithm, 655–659, 661–662 *ex.*
- flying cars, highways for, 850
- FORD-FULKERSON, 686
- Ford-Fulkerson method, 676–693
- FORD-FULKERSON-METHOD, 676
- FORESEE, 797
- forest, 1167, 1169
 - breadth-first, 728
 - depth-first, 564
 - disjoint-set, 527–531
- for**, in pseudocode, 22
 - and loop invariants, 21 *n.*
- fork-join parallelism, 749–770
 - see also* parallel algorithm
- fork-join scheduling, 759–761, 769 *ex.*
- formal power series, 121 *pr.*
- formula satisfiability, 1073–1076
- forward edge, 569
- forward substitution, 822–823
- Fourier transform, *see* discrete Fourier transform
- transform, fast Fourier transform
- fractional knapsack problem, 429
- fractional matching, 741 *pr.*
- free tree, 1167, 1169–1171
- frequency count, 802 *ex.*
- frequency domain, 877
- full binary tree, 1174
 - relation to optimal code, 433
- full node, 502
- full rank, 1220
- full walk of a tree, 1112
- fully parenthesized matrix-chain product, 374
- fully polynomial-time approximation scheme, 1105
 - for subset sum, 1124–1130
- function, 1161–1163
 - Ackermann's, 544
 - α -strongly convex, 1041
 - basis, 841
 - β -smooth, 1041
 - boolean, 1182 *ex.*
 - convex, 1025–1027, 1194
 - driving, 101
 - final-state, 968
 - hash, *see* hash function
 - iterated, 68, 74 *pr.*
 - linear, 30, 853
 - objective, 626, 852, 854
 - potential, 456
 - prefix, 975–977
 - probability distribution, 218 *ex.*
 - quadratic, 31
 - reduction, 1062
 - suffix, 968
 - transition, 967, 973–974
 - watershed, 103
- functional iteration, 68
- fundamental theorem of linear programming, 872
- furthest-in-future, 441
- fusion tree, 226, 478
- fuzzy sorting, 203 *pr.*
- Gabow's scaling algorithm for single-source shortest paths, 641 *pr.*
- gadget, 1086, 1097
- GALE-SHAPLEY, 719
- Gale-Shapley algorithm, 718–722
- Galois field of two elements ($GF(2)$), 1224 *pr.*
- gap character, 961 *ex.*, 975 *ex.*
- gate, 1065
- Gaussian elimination, 825
- gcd, *see* greatest common divisor
- GCD recursion theorem, 911
- general arithmetic series, 1141
- general number-field sieve, 956
- generating function, 121 *pr.*
- generation of partitioned sets, 234
- generator
 - of a subgroup, 922
 - of $\mathbb{Z}_n^*$, 932

- GENERIC-MST, 587
- geometric distribution, 1196–1198
 - and balls and bins, 143–144
 - in finding prime numbers, 943
 - in randomized selection analysis, 232
- geometric series, 1142
- $GF(2)$ (Galois field of two elements), 1224 pr.
- global cut, 701 pr.
- global minimizer, 1022, 1024 fig., 1026 fig.
- global variable, 22
- golden ratio (ϕ), 69, 70 ex.
- gossiping, 475
- gradient descent, 1022–1038
 - constrained, 1032–1034
 - in machine learning, 1035–1037
 - for solving systems of linear equations, 1034–1035
 - stochastic, 1040 pr.
 - unconstrained, 1023–1031
- GRADIENT-DESCENT, 1025
- GRADIENT-DESCENT-CONSTRAINED, 1032
- gradient of a function, 1023
- GRAFT, 542 pr.
- grain size in a parallel algorithm, 783 pr.
- graph, 1164–1169
 - adjacency-list representation of, 550–551
 - adjacency-matrix representation of, 551–552
 - and asymptotic notation, 548
 - attributes of, 548, 552
 - breadth-first search of, 554–563
 - coloring of, 1100 pr.
 - complement of, 1085
 - component, 576
 - constraint, 628–630
 - dense, 549
 - depth-first search of, 563–572
 - dynamic, 523, 817
 - ϵ -dense, 668 pr.
 - hamiltonian, 1056
 - interval, 425 ex.
 - matching in, 693–697, 704–743
 - nonhamiltonian, 1056
 - planar, 584 pr.
 - regular, 716 ex., 740 pr.
 - shortest path in, 558
 - singly connected, 572 ex.
 - sparse, 549
 - static, 522
 - subproblem, 370–371
 - tour of, 1090
 - weighted, 551
 - see also* directed acyclic graph, directed graph, flow network, undirected graph, tree
- GRAPH-ISOMORPHISM, 1060 ex.
- Gray code, 471 pr.
- gray vertex, 554, 564
- greatest common divisor (gcd), 906–907, 910 ex.
 - binary gcd algorithm for, 953 pr.
 - Euclid’s algorithm for, 911–916, 954 pr.
 - with more than two arguments, 916 ex.
 - recursion theorem for, 911
- GREEDY-ACTIVITY-SELECTOR, 424
- GREEDY-BIPARTITE-MATCHING, 726
- greedy-choice property, 427–428
 - of activity selection, 420–421
 - of Huffman codes, 436–437
 - of offline caching, 442–445
- greedy method, 417–447
 - for activity selection, 418–425
 - for coin changing, 446 pr.
 - compared with dynamic programming, 384–385, 393 ex., 421, 426–430
 - Dijkstra’s algorithm, 620–626
 - elements of, 426–431
 - for the fractional knapsack problem, 429
 - greedy-choice property in, 427–428
 - for Huffman code, 431–439
 - Kruskal’s algorithm, 592–594
 - for maximal bipartite matching, 726
 - for minimum spanning tree, 591–599
 - for offline caching, 440–446
 - optimal substructure in, 428
 - Prim’s algorithm, 594–597
 - for set cover, 1115–1119
 - for task-parallel scheduling, 759–761, 769 ex.
 - for task scheduling, 446 pr.
 - for weighted set cover, 1132 pr.
- greedy scheduler, 759
- GREEDY-SET-COVER, 1117
- grid, 697 pr.
- group, 916–923
 - cyclic, 932
 - operator ($\oplus$), 917

- growth step, 736
- guessing the solution, in the substitution method, 92
- Habanero-Java, 750
- half 3-CNF satisfiability, 1099 ex.
- half-open interval ($[a, b)$ or $(a, b]$), 1157
- Hall's theorem, 715 ex.
- halting, 6
- halting problem, 1042
- halving lemma, 887
- HAM-CYCLE, 1056
- hamiltonian cycle, 1043, 1056
 - NP completeness of, 1085–1090
- hamiltonian graph, 1056
- hamiltonian path, 1060 ex., 1098 ex.
- HAM-PATH, 1060 ex.
- handle, 173
- handshaking lemma, 1168 ex.
- harmonic number, 1142, 1149
- harmonic series, 1142, 1149
- HASH-DELETE, 300 ex.
- hash function, 275, 282–292
 - auxiliary, 295
 - collision-resistant, 941
 - cryptographic, 291
 - division method for, 284, 292 ex.
 - for hierarchical memory, 304–307
 - multiplication method for, 284–286
 - multiply-shift method for, 285–286
 - random, 286–290
 - static, 282, 284–286
 - universal, 286–290
 - wee, 305–307
 - see also* family of hash functions
- hashing, 272–311
 - with chaining, 277–281, 308 pr.
 - double, 295–297, 301 ex.
 - independent uniform, 276
 - with linear probing, 297, 302–304
 - in memoization, 368, 391
 - with open addressing, 293–301, 308 pr.
 - perfect, 310
 - random, 286–290
 - to replace adjacency lists, 553 ex.
 - of static sets, 308 pr.
 - of strings, 290–291, 292 ex.
 - uniform, 278
 - universal, 286–290, 309 pr.
 - of variable-length inputs, 290–291
 - of vectors, 290–291
- HASH-INSERT, 294, 300 ex.
- HASH-SEARCH, 294, 300 ex.
- hash table, 275–282
 - dynamic, 470 ex.
 - used within a priority queue, 174
 - see also* hashing
- hash value, 275
- hat-check problem, 134 ex.
- head
 - in a disk drive, 498
 - of a linked list, 259
 - of a queue, 256
- heap, 161–181
 - analyzed by potential method, 459 ex.
 - building, 167–170, 178 pr.
 - in constructing Huffman codes, 436
 - d -ary, 179 pr., 668 pr.
 - deletion from, 178 ex.
 - in Dijkstra's algorithm, 623
 - extracting the maximum key from, 174
 - Fibonacci, 478
 - height of, 163
 - increasing a key in, 174–175
 - insertion into, 175
 - in Johnson's algorithm, 666
 - max-heap, 162
 - maximum key of, 174
 - mergeable, 268 pr.
 - min-heap, 163
 - in Prim's algorithm, 597
 - as a priority queue, 172–178
- HEAP-DECREASE-KEY, 176 ex.
- HEAP-EXTRACT-MIN, 176 ex.
- HEAP-MINIMUM, 176 ex.
- heap property, 162
 - maintenance of, 164–167
 - vs. binary-search-tree property, 315 ex.
- heapsort, 161–181
 - lower bound for, 207
- HEAPSORT, 170
- HEDGE, 1039 pr.
- height
 - black-, 332
 - of a B-tree, 502–504
 - of a d -ary heap, 179 pr.

- height, *continued*
 - of a decision tree, 207
 - of a heap, 163
 - of a node in a heap, 163, 170 *ex.*
 - of a node in a tree, 1173
 - of a red-black tree, 332
 - of a tree, 1173
- height-balanced tree, 357 *pr.*
- helpful partitioning, 232
- Hermitian matrix, 838 *ex.*
- Hessian matrix, 1035
- heuristic
 - first-fit for bin packing, 1131 *pr.*
 - path compression, 528
 - in the Rabin-Karp algorithm, 965
 - for the set-covering problem, 1116, 1132 *pr.*
 - table doubling, 461
 - for the traveling-salesperson problem, 1115 *ex.*
 - union by rank, 528
 - weighted union, 525
- high endpoint of an interval, 489
- high side of a partition, 183
- HIRE-ASSISTANT, 127
- hiring problem, 126–127, 135–136
 - online, 150–152
 - probabilistic analysis of, 132–133
- hit, 965
- HOARE-PARTITION, 199 *pr.*
- HOPCROFT-KARP, 709
- Hopcroft-Karp algorithm, 709–715
- HORNER, 47 *pr.*
- Horner's rule, 46 *pr.*, 879, 963
- HUFFMAN, 434
- Huffman code, 431–439
- Human Genome Project, 6
- HUNGARIAN, 737
- Hungarian algorithm, 723–739, 740 *pr.*
- hybrid cryptosystem, 941
- hyperedge, 1167
- hypergraph, 1167
- hypotheses, 1003

- ideal parallel computer, 756
- idempotency laws, 1154
- identity, 917
- identity matrix, 1215
- identity permutation, 138 *ex.*

- if**, in pseudocode, 22
- ill-defined recurrence, 77
- image, 1162
- image compression, 412 *pr.*
- incidence, 1164–1165
- incidence matrix
 - and difference constraints, 628
 - of a directed graph, 553 *ex.*
- inclusion and exclusion, 1158 *ex.*
- incomplete step, 759
- INCREASE-KEY, 173
- increasing a key, in a max-heap, 174–175
- INCREMENT, 451
- incremental design method, 34
- incrementing, 21
- in-degree, 1165
- indentation in pseudocode, 21–22
- independence
 - of events, 1188, 1190 *ex.*
 - of random variables, 1192
 - of subproblems in dynamic programming, 386–387
- independent family of hash functions, 288
- independent set, 1099 *pr.*
- independent uniform hash function, 276
- independent uniform hashing, 276, 278
- independent uniform permutation hashing, 295
- indexing into an array, 22–23, 26 *n.*, 252
- index of an element of $\mathbb{Z}_n^*$, 933
- indicator random variable, 130–133
 - in approximation algorithm for MAX-3-CNF satisfiability, 1120–1121
 - in birthday paradox analysis, 142–143
 - in bounding the right tail of the binomial distribution, 1207–1208
 - in coin flipping analysis, 131–132
 - expected value of, 130
 - in hashing analysis, 279–281
 - in the hat-check problem, 134 *ex.*
 - in hiring-problem analysis, 132–133
 - and linearity of expectation, 131
 - in quicksort analysis, 197–198, 200 *pr.*
 - in randomized caching analysis, 812
 - in randomized-selection analysis, 245 *pr.*
 - in streak analysis, 148–150
- induced subgraph, 1166
- inequality, linear, 853
- infeasible linear program, 854